Embedded OS Implementation, Fall 2026

Homework #1 (due October 15th, 2026 (Thursday) 08:00)

Hello uCOS-II

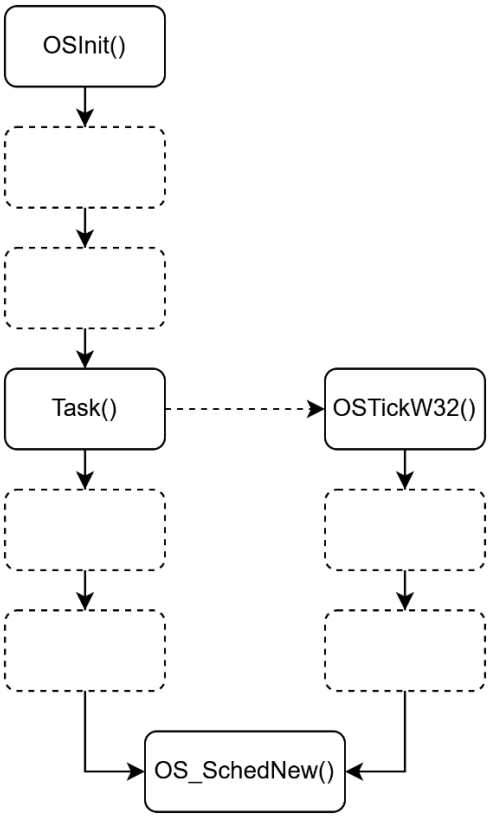
Future PAs will extend from this assignment.

Problem Definition:

(a) Please draw the system flow of "Hello µC/OS-II (the modified main.c in Lab1)" and using a system flow diagram, explain how the kernel starts, schedules, and switches between tasks. **Note: guideline of the system flow diagram shown below, explain by filling in the blanks in the provided diagram with µC/OS-II functions, and also explain what the functions do.**

(b) Consider two periodic tasks (τ_1 ,τ_2), where it uses printf statement to print "**Task(ID) is running** " message from within each task's main loop, then their delay time are 4 ticks and 7 ticks, respectively. Task priority of two tasks (τ_1 ,τ_2) are 1 and 2, respectively. Please add some code to the uCOS-II scheduler **in the kernel level** to observe how CPU is switched among tasks by means of context switches. **Note: Printing format (guidance) are shown in page 2 of this homework.**

System flow diagram example:



Tick	CurrentTask ID	NextTask ID	# of Ctx Switch
##	**********	task(ID)(job number)	#
##	task(ID)(job number)	task(ID)(job number)	#

※ If the task is Idle Task, print "task(priority)".

##	"waiting" or "ready"	prio(task priority)	osrdygrp(0xbefore)(0xafter)	osrdytbl(0xbefore)(0xafter)
##	Task(ID) is running			

※ printing values after osrdygrp and osrdytbl start by "0x", as values are shown in hex values.

```
0       **************          task( 1)( 0)                0
0       task( 1) is running
0       waiting prio( 1)        osrdygrp(0x81)(0x81)    osrdytbl(0x06)(0x04)
0       task( 1)( 0)            task( 2)( 0)                0
0       task( 2) is running
0       waiting prio( 2)        osrdygrp(0x81)(0x80)    osrdytbl(0x04)(0x00)
0       task( 2)( 0)            task(63)                    1
4       ready   prio( 1)        osrdygrp(0x80)(0x81)    osrdytbl(0x00)(0x02)
4       task(63)                task( 1)( 1)                2
4       task( 1) is running
4       waiting prio( 1)        osrdygrp(0x81)(0x80)    osrdytbl(0x02)(0x00)
4       task( 1)( 1)            task(63)                    3
7       ready   prio( 2)        osrdygrp(0x80)(0x81)    osrdytbl(0x00)(0x04)
7       task(63)                task( 2)( 1)                4
7       task( 2) is running
7       waiting prio( 2)        osrdygrp(0x81)(0x80)    osrdytbl(0x04)(0x00)
7       task( 2)( 1)            task(63)                    5
8       ready   prio( 1)        osrdygrp(0x80)(0x81)    osrdytbl(0x00)(0x02)
8       task(63)                task( 1)( 2)                6
8       task( 1) is running
8       waiting prio( 1)        osrdygrp(0x81)(0x80)    osrdytbl(0x02)(0x00)
8       task( 1)( 2)            task(63)                    7
12      ready   prio( 1)        osrdygrp(0x80)(0x81)    osrdytbl(0x00)(0x02)
12      task(63)                task( 1)( 3)                8
12      task( 1) is running
12      waiting prio( 1)        osrdygrp(0x81)(0x80)    osrdytbl(0x02)(0x00)
12      task( 1)( 3)            task(63)                    9
14      ready   prio( 2)        osrdygrp(0x80)(0x81)    osrdytbl(0x00)(0x04)
14      task(63)                task( 2)( 2)                10
14      task( 2) is running
14      waiting prio( 2)        osrdygrp(0x81)(0x80)    osrdytbl(0x04)(0x00)
14      task( 2)( 2)            task(63)                    11
16      ready   prio( 1)        osrdygrp(0x80)(0x81)    osrdytbl(0x00)(0x02)
16      task(63)                task( 1)( 4)                12
16      task( 1) is running
```

Crediting :

Your homework need to show the following information.

- The system flow and the explanation of the process (functions). **(45%)**
- The result (Output.txt). **(10%)**
- A report that describes your implementation (please attach the screenshot of the code and **Mark** the modified part). **(45%)**

Hints:

1. Call the function **OSTimeSet(0)** before the OS starts to initialize the start time.
2. Use **OSTimeGet()** to get the current tick in the system.
3. Use '**/t**' to space out your code prints.
4. If your project size is too large for uploading, you can try to delete the ".vs" or the "Debug" Folders
5. The task example

```
static void task(void* p_arg) {
    /*
     ...
     */
    while (1) {
        /******************************** HW 1 *********************************/
        // print task running message here !!!
        /***********************************************************************/
    }
    /*
     ...
     */
}
```

Homework submitting format and rules:

Submit to Moodle

Submit deadline : October 15th, 2026 (Thursday) 08:00

File name format : RTOS_ your student ID_HW1.zip

RTOS_ Student ID_HW1.zip **includes** :

※ The report (RTOS_ your student ID_HW1.**pdf**).

※ Folder with executable µC/OS-II project (**RTOS_ your student ID _HW1**).

※ Standard input and output filenames in the project are necessary for the checker, please check before submitting.

```
#define INPUT_FILE_NAME "./TaskSet.txt"
#define OUTPUT_FILE_NAME "./Output.txt"
```

※ Plagiarizing is strictly prohibited.

※ RTOS_Myyyddxxx_HW1.zip must be including files as follow:

